



BLOCK TEAM

블록 코딩 로봇 교육 플랫폼

ROS 2 Humble 기반 TurtleBot3 블록 코딩 교육 플랫폼





목차

프로젝트 개요

..... 01

프로젝트 소개

..... 02

개발 목표

..... 03

시스템 구성

..... 04

주요 구현 기능

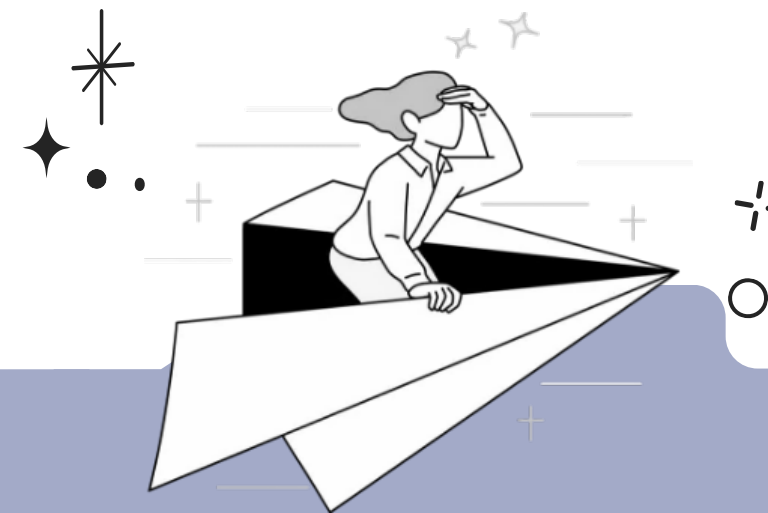
..... 05

결론 및 향후 계획

..... 06



프로젝트 개요



개발 환경 및 기간

- 개발 기간: 5일 (40시간)
- 개발 환경: Ubuntu 22.04 LTS
- ROS 버전: ROS 2 Humble
- 하드웨어: TurtleBot3 Burger
- 개발 언어: Python, HTML5, CSS3, JavaScript

본 프로젝트는 웹 기반 블록 코딩 환경과 ROS 2 기반 TurtleBot3를 연동하여 초등학생도 쉽게 사용할 수 있는 교육용 로봇 제어 플랫폼을 구현하는 것을 목표로 하였습니다.

하드웨어 및 언어



프로젝트 소개

BLOCK TEAM

웹 기반 블록 코딩 환경

- 사용자는 웹 브라우저에서 블록을 조합하여 프로그램 작성
- Drag & Drop 방식의 직관적인 인터페이스 제공
- 초등학생도 쉽게 사용 가능한 교육용 환경 구축

JSON Tree 기반 프로그램 변환

- 실행 버튼을 누르면 블록 프로그램이 JSON Tree 형태로 변환
- 구조화된 데이터 형식으로 프로그램 표현
- Runtime Interpreter가 해석 가능한 형태로 전달

rosbridge_server를 통한 ROS 2 연동

- JSON Tree가 rosbridge_server를 통해 ROS 2로 전달
- WebSocket 기반 실시간 양방향 통신 구현
- Interpreter Node가 런타임에 해석하여 TurtleBot3 제어



개발 목표

BLOCK TEAM



블록 코딩 환경

- 웹 기반 블록 코딩 환경 구축
- 직관적인 UI 제공



실시간 제어

- TurtleBot3 실시간 제어
- 웹과 로봇 상태 동기화



Interpreter 구현

- ROS2 기반 Runtime Interpreter 구현
- JSON Tree 실시간 해석



중첩 구조 지원

- 반복문 및 조건문의 중첩 구조 지원
- Stack 기반 파싱



안전 주행

- LiDAR 기반 안전 주행
- 장애물 감지 및 대응



전체 시스템 아키텍처

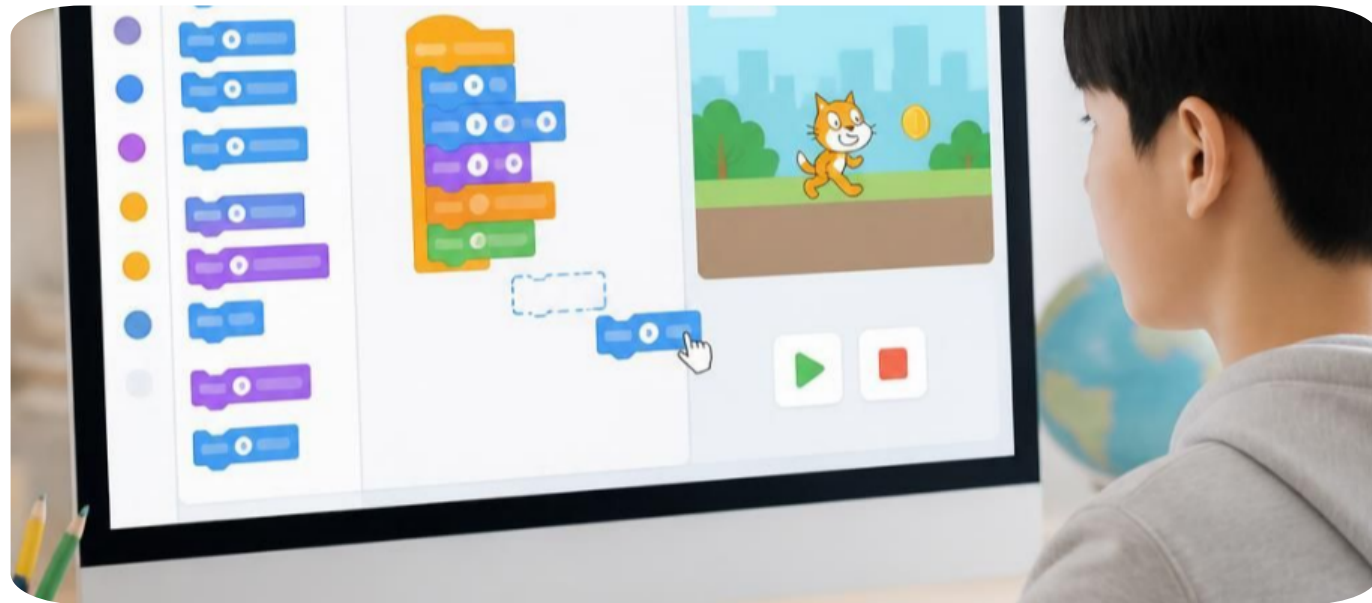


- Web Browser: 사용자가 블록을 조합하여 프로그램 작성
- rosbridge_server: WebSocket 기반으로 웹과 ROS 2 연결
- Interpreter Node: JSON Tree를 실시간으로 해석하여 로봇 제어
- cmd_vel Topic: 로봇의 속도 명령 전달
- run_state Topic: 현재 실행 중인 블록 상태 전달
- buzzer Topic: 부저 제어 명령 전달
- Ultrasonic Node: LiDAR 데이터 기반 장애물 감지
- TurtleBot3: 실제 로봇 하드웨어 제어 및 동작 수행



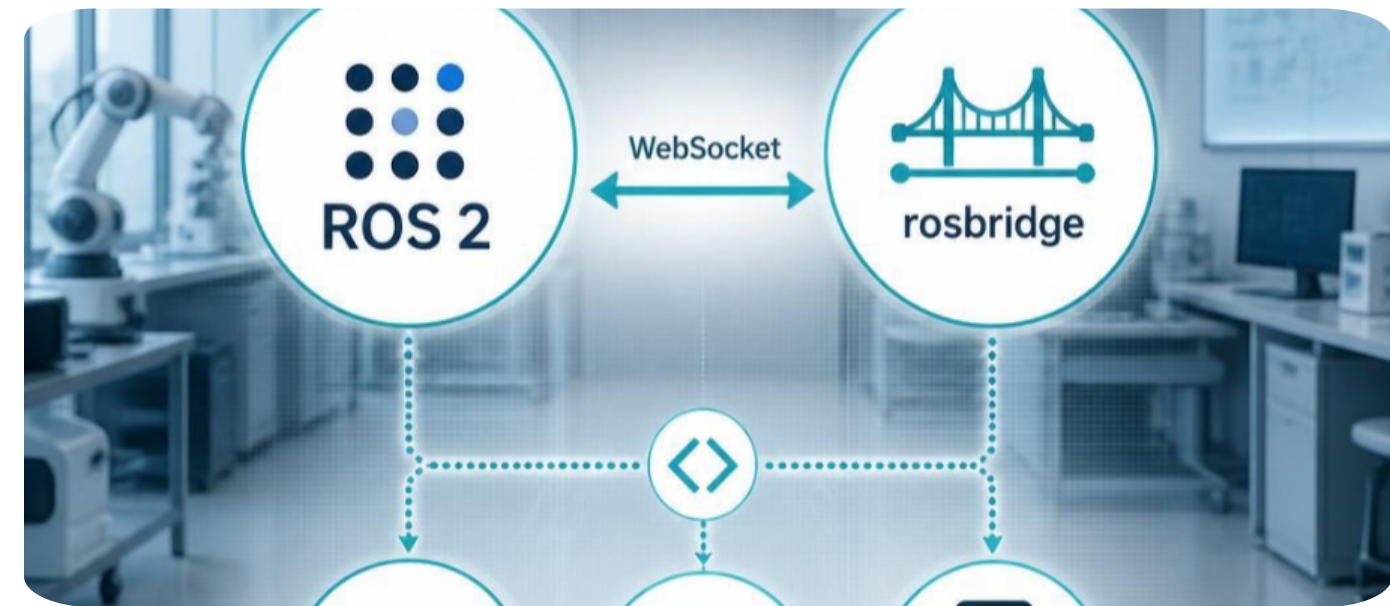
주요 구현 기능 (1)

BLOCK TEAM



블록 코딩 UI

- Drag & Drop 방식의 블록 편집기
- 프로그램 실행 및 정지 기능
- 현재 실행 블록 하이라이트
- 직관적인 사용자 인터페이스



ROS2 연동

- rosbridge_server 기반 WebSocket 통신
- /program, /cmd_vel, /run_state Topic 활용
- 웹과 ROS2 간 실시간 데이터 교환
- 안정적인 양방향 통신 구현



주요 구현 기능 (2)

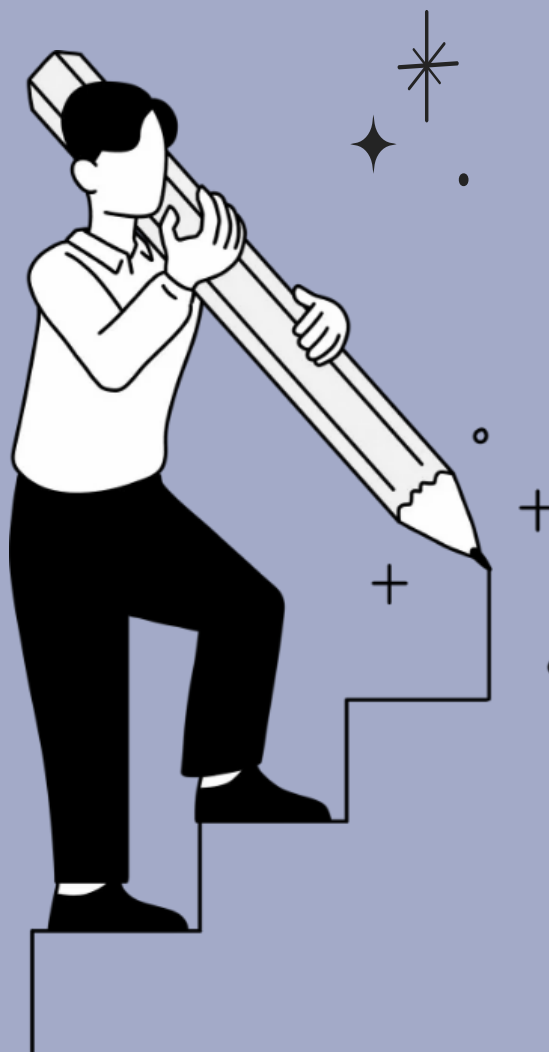
BLOCK TEAM

JSON Tree 기반 Runtime Interpreter

- 블록 프로그램을 JSON Tree로 변환
- 런타임에 실시간 해석 및 실행
- TurtleBot3 제어 명령 생성

반복문 및 조건문 중첩 지원

- 반복문(Loop) 구조 지원
- 조건문(If-Else) 구조 지원
- 중첩된 블록 구조 완벽 처리



Stack 기반 Runtime Parsing

- Stack 구조 기반 프로그램 파싱
- 효율적인 메모리 관리
- 복잡한 제어 흐름 처리

실시간 상태 동기화

- 웹과 로봇의 실시간 상태 동기화
- 현재 실행 블록 정보 전달
- WebSocket 기반 양방향 통신



LiDAR 기반 안전 기능

BLOCK TEAM

전방·후방 장애물 감지

- LiDAR 센서를 활용한 실시간 장애물 탐지
- 전방 및 후방 방향 동시 감지
- 안전 거리 기반 위험 판단

Runtime Skip 구조

- 장애물 감지 시 현재 이동 블록만 중단
- 다음 실행 흐름으로 자연스럽게 이어짐
- 프로그램 전체 종료 방지

Emergency Stop 기능

- 위험 상황 시 즉시 정지
- 부저를 통한 경고음 발생
- 안전한 로봇 운용 보장



역할 분담 및 개발 과정

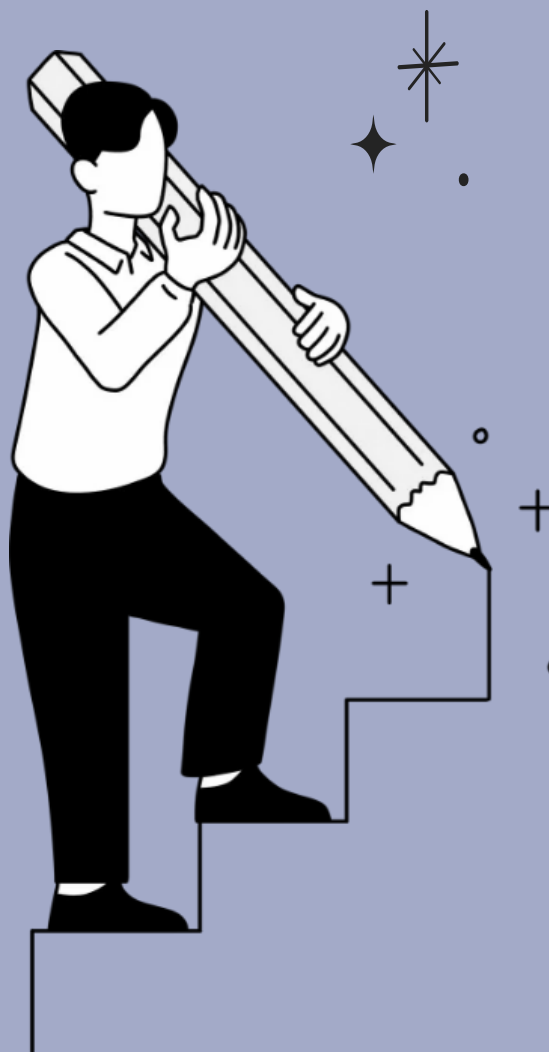
BLOCK TEAM

팀 구성 및 역할

- 박정호: Full Stack 개발
- 웹 UI, ROS2 연동, Interpreter
- 시스템 통합 및 문서화

주요 개발 내용

- Day 1-2: 설계 및 기본 구현
- Day 3-4: 통합 및 기능 확장
- Day 5: 안정화 및 문서 작성



개발 일정

- 총 5일 (40시간) 개발 기간
- 단계별 개발 진행
- 일정 관리 및 프로젝트 총괄

통합 및 테스트

- 전체 시스템 통합 테스트
- 사용자 테스트 수행
- 빌드 및 배포 준비



트러블슈팅 및 테스트 결과

BLOCK TEAM

주요 문제 해결

- WebSocket 연결 실패: rosbridge_server 실행 절차 추가
- Highlight 불일치: /run_state 기반 갱신으로 해결
- 중첩 블록 오류: Runtime Interpreter 개선

시스템 개선

- Runtime Skip 적용으로 장애물 감지 시 흐름 유지
- 실행 중 프로그램 수정 방지를 위한 UI Lock 추가
- Stack 처리 로직 개선

테스트 결과

- 모든 필수 기능 정상 동작 확인
- 웹 실행, ROS2 통신, TurtleBot3 제어 성공
- 반복문, 조건문, 중첩 구조 모두 검증 완료



결론 및 향후 계획

BLOCK TEAM

- 웹 기반 블록 코딩과 ROS 2 TurtleBot3 연동 교육 플랫폼을 구현하였다.
- 향후 3D 시뮬레이터 및 다양한 ROS2 로봇 지원으로 확장할 계획이다.

